ITM SKILLS UNIVERSITY

School of Future Tech

Case Study Report

On

Async Username Validator

By

SAAD YUSUF KHAN

150096725157

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

---

1. Introduction to the Case Study

In modern web applications, username validation is a critical user experience feature. When users sign up for a platform, they need to know in real time whether their chosen username is available or already taken. Traditional synchronous validation requires page reloads, leading to a poor user experience.

This case study focuses on designing and implementing an **Async Username Validator** that checks username availability asynchronously using JavaScript Promises and the async/await syntax.

The case study demonstrates how front-end developers can simulate server-side validation using mock data and asynchronous JavaScript. It highlights the importance of Promise-based workflows, async/await, input validation, and real-time UI feedback in building practical and responsive web interfaces.

---

2. Problem Statement / Case Background (Abstract)

Background

Traditional form validation submits data to the server and waits for a response, causing unnecessary page reloads. Modern applications demand asynchronous validation that checks availability instantly without disrupting the user flow. JavaScript's async/await and Promise API enable developers to handle such tasks elegantly on the client side.

Abstract

This case study presents the design and implementation of an **Async Username Validator** using vanilla JavaScript and the async/await pattern. A mock database of existing usernames is checked asynchronously with a simulated network delay using *setTimeout* wrapped in a Promise. The system validates user input (empty check, minimum length), shows a loading state, and then displays whether the username is available or taken. The UI is built with HTML and CSS, providing clear visual feedback through color-coded messages.

---

3. Case Study Design

The Async Username Validator is designed as a simple single-page application with the following pipeline stages:

1. **User Input**
    - User types a desired username into an input field.
    - Click the 'Check Availability' button to trigger validation.
2. **Client-side Validation**
    - Check if the input is empty → show warning.
    - Check minimum length (at least 3 characters) → show warning.
3. **Async Availability Check**
    - Show a loading message: "Checking availability..."
    - Wait for a simulated async response (1500ms delay using Promise + setTimeout).
4. **Result Display**
    - If username exists in the mock database → show error in red.
    - If username is free → show success in green.

---

4. Methods & Algorithms / Technology Applied in the Problem Statement

Key methods and algorithms:

1. **JavaScript Async/Await**
    - The async function validateUsername() uses await to pause execution until the Promise resolves or rejects.
    - Provides clean, readable code without callback hell.
2. **Promise API**
    - checkUsernameAvailability() returns a Promise.
    - resolve() is called when the username is available.
    - reject() is called when the username is already taken.
3. **Input Validation**
    - Empty input check using .trim() === ''.
    - Minimum length check (username.length < 3).
4. **Mock Database Simulation**
    - Array of predefined usernames acts as a mock server database.
    - .includes(username.toLowerCase()) performs case-insensitive lookup.
5. **DOM Manipulation**
    - Real-time feedback by updating textContent and style.color of the message element.

Technology Stack Used for the Case

- Programming language: JavaScript (Vanilla JS, ES6+).
- Markup: HTML5.
- Styling: CSS3 with Flexbox and gradient backgrounds.
- Environment: Any modern browser; VS Code or any text editor.
- Data storage: In-memory JavaScript array (mock database).

---

5. Problem Statement / Case Study Implementation Details and Snapshots

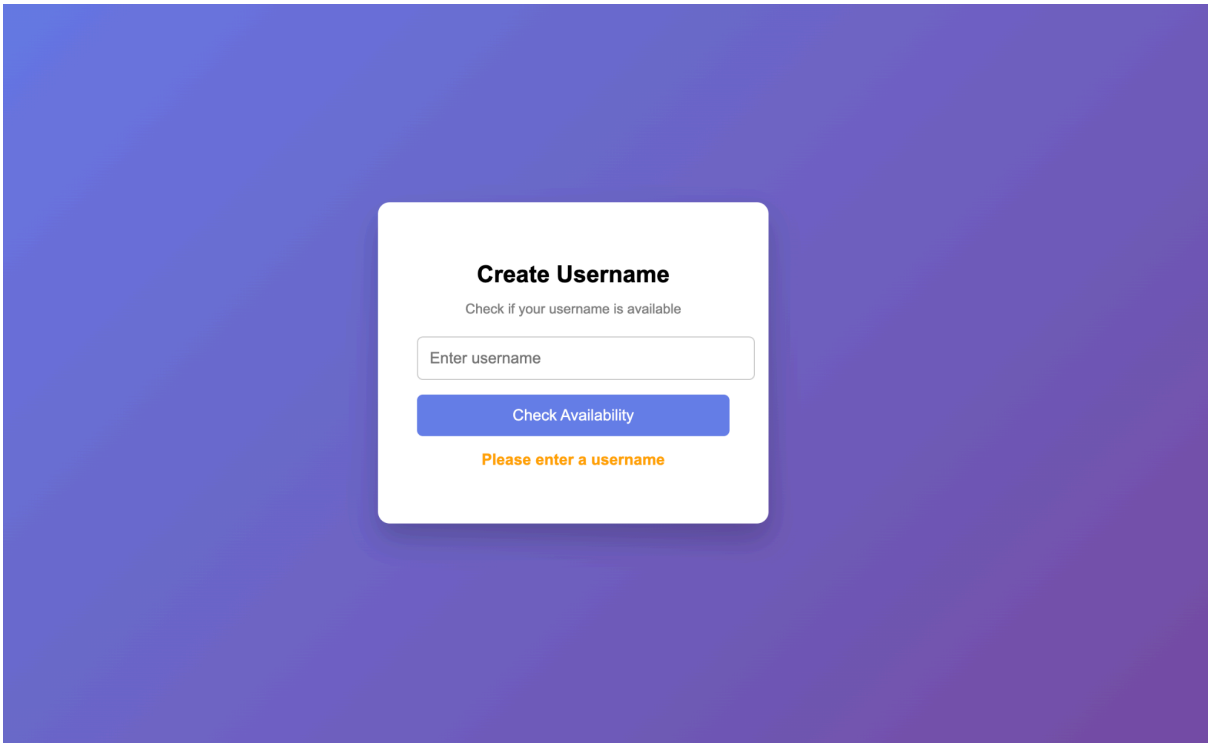
The implementation is organized across three files:

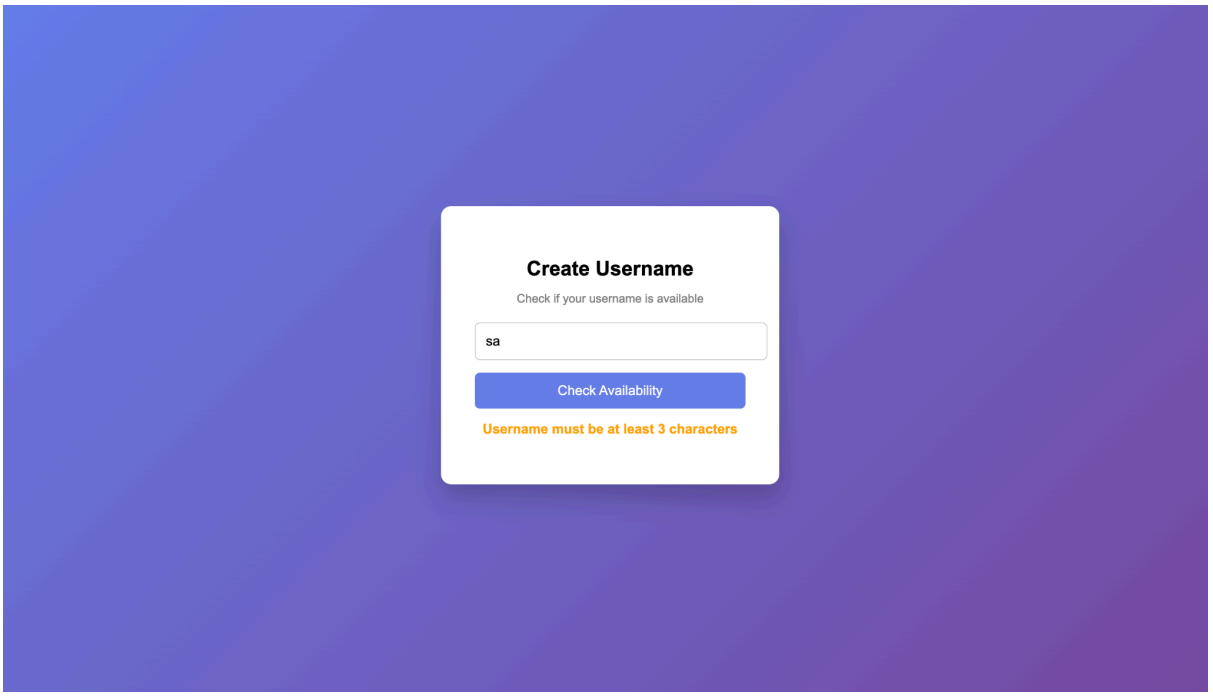
1. **index.html** — Defines the UI structure: a card with heading, subtitle, input field, button, and message paragraph.
2. **style.css** — Styles the card using Flexbox centering, gradient background, rounded inputs and buttons, and color feedback.
3. **script.js** — Contains the mock database array, async validation function, Promise-based availability checker, and event listener.

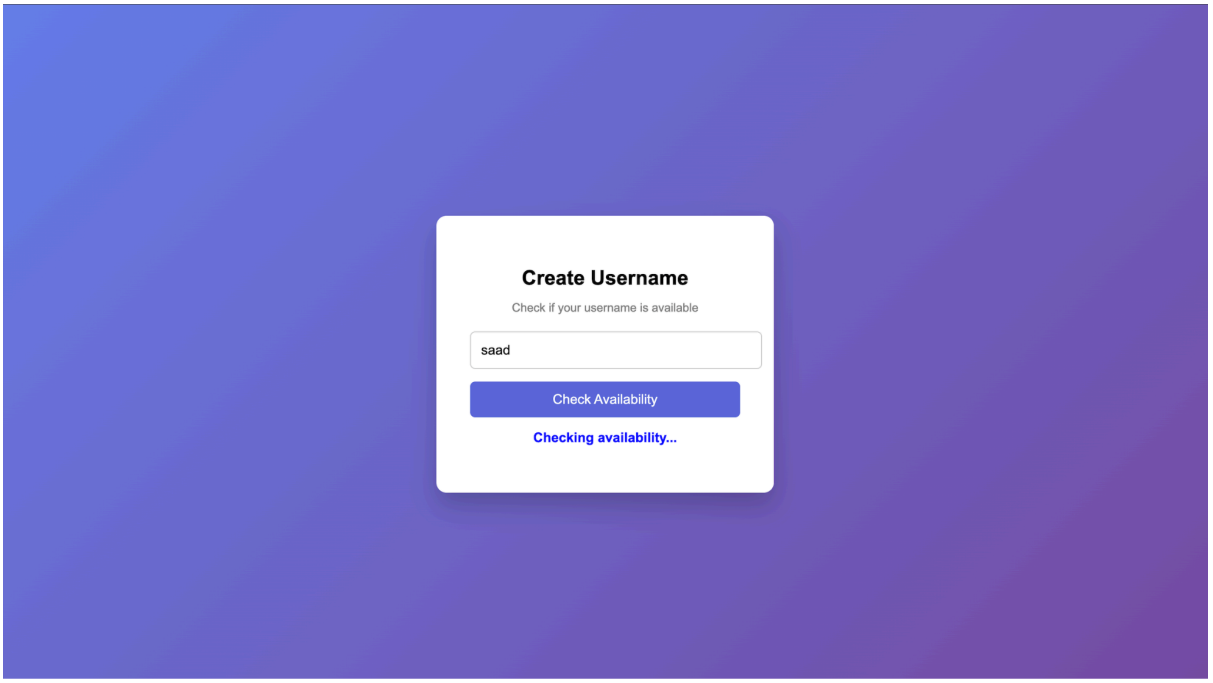
Code Snapshot – script.js

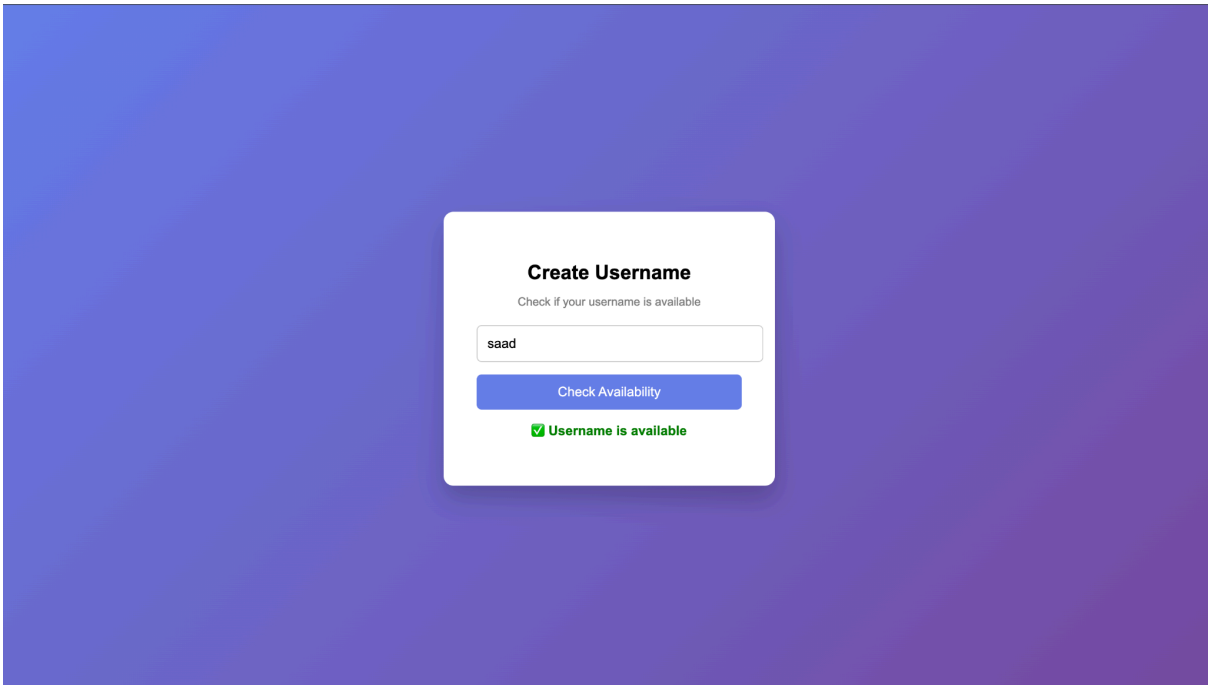
```
const users =
["admin","john123","prasad","alex","emma","david","mike","rohan","akash","raj"]
```

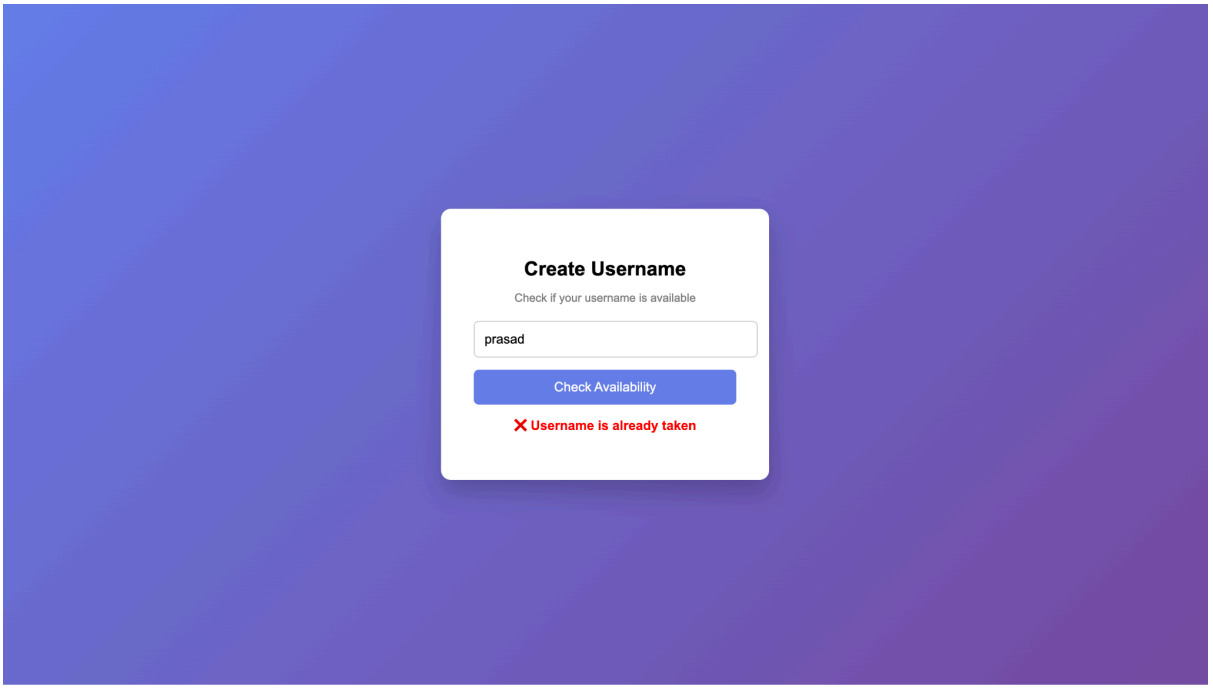
```
function checkUsernameAvailability(username){
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if(users.includes(username.toLowerCase())){
        reject("Username is already taken")
      } else {
        resolve("Username is available")
      }
    }, 1500)
  })
}

async function validateUsername(){
  const username = input.value.trim()
  if(username === ""){
    message.textContent = "Please enter a username"
    message.style.color = "orange"; return
  }
  if(username.length < 3){
    message.textContent = "Username must be at least 3 characters"
    message.style.color = "orange"; return
  }
  message.textContent = "Checking availability..."
  message.style.color = "blue"
  try {
    const result = await checkUsernameAvailability(username)
    message.textContent = result
    message.style.color = "green"
  } catch(error) {
    message.textContent = error
    message.style.color = "red"
  }
}
button.addEventListener("click", validateUsername)
```


Create Username
Check if your username is available
Enter username
Check Availability
Please enter a username


Create Username
Check if your username is available
sa
Check Availability
Username must be at least 3 characters


Create Username
Check if your username is available
saad
Check Availability
Checking availability...


Create Username
Check if your username is available
saad
Check Availability
✅ Username is available



---

6. Problem Statement / Case Study Results and Conclusion

Findings

- Using Promises with async/await produces clean, readable asynchronous code without callback nesting.
- Client-side input validation (empty check, minimum length) improves UX by preventing unnecessary async calls.
- Simulating network delay with setTimeout accurately mimics real server response behavior for testing.
- Color-coded UI feedback (orange, blue, green, red) gives users instant, clear status information.
- Case-insensitive username lookup using .toLowerCase() ensures consistent matching regardless of input casing.

Conclusion

This case study demonstrates a complete pipeline for building an Async Username Validator using HTML, CSS, and vanilla JavaScript. It integrates client-side validation, Promise-based asynchronous logic, simulated network delay, and real-time DOM feedback into a cohesive user experience. The implementation serves as a strong foundation for integrating real back-end API calls for username availability in production applications.

---

7. References

1. MDN Web Docs – JavaScript Promises: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise
2. MDN Web Docs – async function: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
3. MDN Web Docs – DOM Manipulation and Events.
4. W3Schools – HTML Forms and CSS Flexbox documentation.
5. Academic tutorials on asynchronous JavaScript patterns and front-end form validation best practices.